REQUIREMENT MANAGER + GITHUB COPILOT · PILOT PROPOSAL

Auditing a repository that is already traced

A pilot to find ambiguities, gaps and bugs in a repository whose requirements already have code, tests and trace tags.

Method and worked example. Figures from a real run are reported after the pilot, against the plan on slides 11 and 19.

BEFORE WE START

Terms used in this deck

TERM	MEANING
Requirement	A Markdown file with an id, a `status` and binding clauses.
Clause	One binding sentence: “returns true when the pedal is above 5 %”.
Case	A Given / When / Then example that proves a clause: `CASE-1`.
Trace tag	A comment linking code to a requirement: `// implements: ID` in the code, `// tested-by: ID` on a test, `// verifies: ID#CASE-1` on the test of one case.
Confirmed	The status a person sets after reading a requirement.
Baseline · **drift**	`sync` stores a hash of each confirmed text. If the text changes later, that is drift: rule **RM018**.
MCP	Model Context Protocol: lets Copilot call the tool as functions and get JSON back.
Agent mode	The Copilot mode that opens files and, with approval, runs commands by itself.
False positive	A finding that turns out to be fine on inspection.

THE METHOD

Three kinds of problem, three layers

LAYER	HOW IT WORKS	WHAT IT CONTRIBUTES
1 · The tool	Deterministic, whole corpus, seconds. Same result on every run.	**Gaps:** clauses with no test, cases nothing verifies, code no requirement claims. **Ambiguities:** vague terms, no failure case.
2 · Copilot	Judgement, on a selection, guided by layer 1.	**Bugs:** code that does something other than the clause, tests that do not prove their case.
3 · A person	The decision. Every finding ends in exactly one outcome.	real bug fix + test ambiguity owner rewords gap add the test false positive counted

Only a person changes a requirement or sets its status. Neither the tool nor Copilot decides that something is a bug.

DATA AND PRIVACY

The engine opens no network connection

No network

No engine module imports `urllib`, `http`, `socket`, `ssl` or `requests`; its only subprocesses are `git` and itself. No telemetry, account or licence server. Runs offline.

Local git only

`log`, `diff`, `show`, `status`, `ls-files`, `rev-parse`, `config`, `tag`. Never `fetch`, `pull`, `push` or `clone`.

Writes only inside the repository

`requirements/` on `sync`; release files only on `sync --release --apply`; config files only on `init`. `gate`, `clarify`, `ask` write nothing. MCP runs over stdio, opens no port.

the exception

Copilot

In agent mode Copilot can open any file in the workspace and, with approval, run terminal commands, not only the files the tool names. All of it goes to the Copilot service under the organisation's Copilot agreement and policies. Check those policies before the pilot, including content exclusion and whether it applies to agent mode. Where that is not allowed: run layer 1 only.

The optional HTML map viewer requests its web fonts from Google Fonts when opened online; the request carries no repository content.

AT SCALE

Hundreds of requirements: the funnel

All requirements
layer 1, automatic

Ranked by risk signals
ordering key, right

Ranked set + random set
reviewed the same way

Copilot findings
layer 2, two runs each

After triage
layer 3

Ordering key, from `gate --risk --json`: untested cases first, then the number of `clarify` questions, then linked files; ties by id.

By family: one architecture requirement with its children per conversation, so related clauses share context.

Per requirement: `gate --show ID --json` (in VS Code `reqmap_show`) is the dossier Copilot reads. Slide 8 shows one.

Random set: drawn from everything not ranked, with a seed written down before the first review. Without it, results say nothing about the rest.

LAYER 1

What the tool finds on its own

COMMAND	WHAT IT FINDS
`gate`	Tags pointing at ids that do not exist, confirmed requirements with no code, broken frontmatter, a requirement whose text changed since the last `sync` recorded its hash (RM018)
`gate --risk`	Requirements with no tests, cases no test verifies, code no requirement claims
`clarify`	Vague terms with no threshold, clauses with no case, no failure path, oversized requirements

Also: `gate --full` (readability), `ask --dupes` (overlapping requirements), `gate --audit` (everything). All have `--json`.

on audit day **RM018 is empty**

On an existing repository the first `sync` is the baseline: drift is measured from that moment on. Whatever drifted before is found only by layer 2. From the pilot on, RM018 catches every new change.

LAYER 2

Copilot hunts bugs

For each requirement Copilot reads the dossier (`reqmap_show`), opens the files it names, and answers one prompt:

```
For each clause of this requirement: show the line of code that does it and the test
that proves it. Say where the code does something other than the clause, where there
is no code for it, and where the test does not actually check the case.
```

bug code does something other than the clause not implemented clause with no code false coverage test does not prove the case

Two runs per requirement, in fresh conversations. A verdict that changes between runs is *unresolved*, never a bug.

LAYER 2 · WHAT COPILOT RECEIVES

The dossier, as JSON

```
$ reqmap.py gate --show EXL-BRAKELIGHT-001 --json
{
  "id": "EXL-BRAKELIGHT-001",
  "status": "confirmed",
  "title": "Brake light follows the brake pedal",
  "intent": "The brake light is the only signal the
             driver behind gets that this car is
             slowing down.",
  "contract": [
    "`brakeLightOn(pedalPercent)` returns true when
     the pedal position is above 5 %, and false
     otherwise.",
    "A pedal reading below 0 %, above 100 % or not a
     number is a sensor fault, and the light turns on."
  ],
  "members": [
    {"role": "implements", "file": "src/brake_light.cpp", "line": 1},
    {"role": "implements", "file": "src/brake_light.h",   "line": 1},
    {"role": "tested-by",  "file": "tests/test_brake_light.cpp", "line": 1}
  ],
  "body": "… ## Cases\nCASE-1 — a pressed pedal …"
}
```

contract · the clauses to check, one by one.

members · where to look: every file tagged to this requirement, with the line of its tag. Copilot opens these, not the whole repository.

body · the full text, with the cases.

status · only confirmed requirements are audited.

Real output of the engine on the worked example, shortened: some empty fields and the frontmatter copy are left out.

Read-only MCP, plus audit instructions

1 · The tool, as a read-only MCP server

`.vscode/mcp.json` (written by `init`, or by hand); the server is started **without** `--allow-writes`:

```
{"servers": {"reqmap": {"type": "stdio",
  "command": "python",
  "args": ["${workspaceFolder}/scripts/reqmap.py", "mcp",
    "--root", "${workspaceFolder}",
    "--reqs", "${workspaceFolder}/requirements",
    "--code", "${workspaceFolder}"]}}}
```

Copilot gets **11 read-only tools**: `reqmap_show`, `reqmap_clarify`, `reqmap_review`, `reqmap_audit`, `reqmap_gate` … with typed arguments and JSON answers. `reqmap_sync` and `reqmap_release` exist only with `mcp --allow-writes`, which the pilot does not use.

2 · The method, as instructions

A short `.github/copilot-instructions.md` for the audit:

- never edit a requirement or its `status:`;
- use the review prompt from slide 7, one requirement at a time;
- two runs per requirement, each in a fresh conversation;
- label every finding bug, not implemented or false coverage, and nothing else.

The full requirement-manager skill teaches authoring (draft, sync, confirm), which writes. Worth installing only if the team later uses the tool to maintain requirements.

LAYER 3

Every finding lands in exactly one counter

COPILOT OR THE TOOL SAYS	A PERSON DECIDES	COUNTER
bug	real: the code does not do what the clause says	**B** real bugs
bug	not a bug	$\mathbf{F_{copilot}}$
not implemented / false coverage	correct behaviour, nothing proves it	**G** gaps
any	the requirement itself can be read two ways	**A** ambiguities
`clarify` question	not a real question	$\mathbf{F_{tool}}$
OK (random set only)	a bug is there anyway	**M** missed
different in the two runs	—	**U** unresolved

In the random set a person checks the OK verdicts too. That is the only way to count what Copilot misses.

MEASUREMENT PLAN

Fixed before the first review

Sample

- **25 ranked** requirements, top of the ordering key.
- **25 random** from the rest; seed recorded before review.
- Copilot runs **k = 2** per requirement; agreement rate reported.

Reported as k/n with a Wilson 95% interval

- Copilot precision = B / (B + $F_{copilot}$)
- Miss rate on the random set = M / 25
- Tool question precision = real / asked

Small numbers stay small

0 real bugs in 25 random requirements means **below about 13 %** at 95 %, not “no bugs”. Any rate from fewer than 5 positives is reported, not interpreted.

Decision bands (proposed, agreed before the run)

- continue Copilot precision lower bound ≥ 50 % and at least 1 real bug or ambiguity per 10 requirements reviewed
- change the method precision lower bound below 50 %
- stop no finding survives triage in the first 20

THE PILOT, STEP BY STEP

What happens, in order

#	STEP	WHO	HOW
0	Check the Copilot policy; copy the engine into `scripts/`; add `.vscode/mcp.json`	lead	slides 4 and 9
1	Baseline	tool	`python scripts/reqmap.py sync`
2	Layer 1 counts	tool	`gate --risk --json`, `clarify --json`
3	Pick 25 ranked + 25 random; write the seed in the pilot log first	lead	ordering key, slide 5
4	Run 1: one requirement per fresh conversation	Copilot	`reqmap_show` + the prompt
5	Run 2: the same, in a new conversation	Copilot	as step 4
6	Triage: each finding into one counter	person	slide 10
7	Rates as k/n with intervals, against the bands; refresh the map	lead	slide 11; `sync` writes `_map.html`

Planned for the next version of the tool, not available yet: steps 3, 6 and 7 recorded and computed by the tool itself, and an Audit view in `_map.html`.

The baseline: a confirmed requirement

Requirement `EXL-BRAKELIGHT-001` · status: confirmed

```
- `brakeLightOn(pedalPercent)` returns true when the
  pedal position is above 5 %, and false otherwise.

CASE-1 — a pressed pedal turns the light on
  Given 20 %   When brakeLightOn(20.0f)  Then true
CASE-2 — a released pedal keeps the light off
  Given 0 %    When brakeLightOn(0.0f)   Then false
CASE-3 — the threshold itself is off
  Given 5 %    When brakeLightOn(5.0f)   Then false
```

An engineer reads it and sets `status: confirmed`. `sync` stores a hash of its text: this is the version everyone agreed on.

`src/brake_light.cpp`

```
// implements: EXL-BRAKELIGHT-001
bool brakeLightOn(float pedalPercent) {
    return pedalPercent > 5.0f;
}
```

`tests/test_brake_light.cpp` · GoogleTest

```
// tested-by: EXL-BRAKELIGHT-001
TEST(BrakeLight, PressedPedalTurnsLightOn) {  // CASE-1
    EXPECT_TRUE(brakeLightOn(20.0f)); }
TEST(BrakeLight, ReleasedPedalKeepsLightOff) { // CASE-2
    EXPECT_FALSE(brakeLightOn(0.0f)); }
TEST(BrakeLight, ThresholdIsOff) {             // CASE-3
    EXPECT_FALSE(brakeLightOn(5.0f)); }
```

not executed by reading the code, all three would pass. gate: PASS (real run).

WORKED EXAMPLE 2 / 6

The change: a safety clause is added

A safety review asks for a fail-safe. The requirement owner adds one clause:

```
  - `brakeLightOn(pedalPercent)` returns true when the pedal position is above 5 %,
    and false otherwise.
+ - A pedal reading below 0 %, above 100 % or not a number is a sensor fault,
+   and the light turns on.
```

The code

Not touched. It was written for the old text.

The tests

Not touched. Reading the code, all three would still pass not executed

CI without the tool

Green. Nothing tells anyone that the code is now behind the requirement.

What the tool reports, line by line

```
$ reqmap.py gate
WARN  RM018 EXL-BRAKELIGHT-001: DRIFT — contract
      changed since lock; re-check 3 member(s):
      src/brake_light.cpp:1, src/brake_light.h:1,
      tests/test_brake_light.cpp:1

$ reqmap.py clarify EXL-BRAKELIGHT-001
ADVISORY
  [no-failure-case · wording] Cases
  What happens on the failure path -- missing
  input, invalid value, nothing to do?
```

Real output, copied from a run of the engine on this example.

RM018 · the text changed since the last `sync` recorded its hash. Here the baseline predates the change; on audit day it would not.

re-check 3 member(s) · every file tagged to this requirement: the review list.

no-failure-case · none of the three cases describes something going wrong, so the fault clause has no case.

not said whether the code is wrong.

Hypothesis: what Copilot should find

PEDAL READING	REQUIREMENT SAYS	CODE RETURNS	VERDICT
`20.0f`	on	on	OK · CASE-1
`0.0f`	off	off	OK · CASE-2
`5.0f`	off	off	OK · CASE-3
`-3.0f` (sensor fault)	on	**off**	bug
`NAN` (sensor fault)	on	**off** · `NAN > 5.0f` is false	bug
`150.0f` (sensor fault)	on	on, only because 150 > 5	right by accident

bug on a pedal sensor fault the brake light stays off · false coverage the three tests say nothing about the fault clause

Expected answer, derived from the code. Not a recorded Copilot run; in the pilot each requirement gets two real runs.

A person decides, and fixes

real bug safety-relevant: handle the fault before the threshold.

```
bool brakeLightOn(float pedalPercent) {
    if (std::isnan(pedalPercent) || pedalPercent < 0.0f
            || pedalPercent > 100.0f) {
        return true;  // sensor fault: fail safe, light on
    }
    return pedalPercent > 5.0f;
}
```

gap a fourth case, and its test:

```
TEST(BrakeLight, SensorFaultTurnsLightOn) {  // CASE-4
    EXPECT_TRUE(brakeLightOn(-3.0f));
    EXPECT_TRUE(brakeLightOn(150.0f));
    EXPECT_TRUE(brakeLightOn(NAN));
}
```

Fix and test shown as written; not compiled.

ambiguity **a question for the requirement owner**

How long does the light stay on after a fault? While the reading is bad, or latched until ignition off?

The requirement does not say, and the two answers are different code. Neither Copilot nor the tool should pick one: the owner decides and writes it into the requirement.

After the fix

```
$ reqmap.py sync --accept-drift "Sensor-fault clause
    implemented and tested (CASE-4)"
  accepted drift on 1 confirmed contract(s), reason:
  Sensor-fault clause implemented and tested (CASE-4)
lock updated.

$ reqmap.py gate
gate: PASS

$ reqmap.py clarify EXL-BRAKELIGHT-001
  [no-failure-case · wording] Cases
  What happens on the failure path -- ...
```

Real output, copied from the run on this example.

accept-drift with a reason · the new text becomes the agreed version, and the reason is kept in the drift log: who accepted what, and why.

gate: PASS · only records that the change was accepted. It is the **CASE-4 test** that proves the fix.

F_{tool} `clarify` still asks for a failure case, although CASE-4 is one: its word list (`CLARIFY_FAILURE_WORDS`, `clarify.py:52`) lacks “fault” and is not configurable. Counted as a tool false positive; fixing it is an engine change, planned.

THE PILOT

Scope, cost and when to stop

Scope

50 requirements: 25 ranked + 25 random.
Two Copilot runs each.

Cost

Layer 1: under 1 person-hour of setup, seconds per run.
Layer 2: existing Copilot seats.
Layer 3: 15–30 min per requirement, **capped at 30 person-hours**.

Stop rule

Stop after the first **20** reviewed if no finding survives triage, or when the **30-hour** cap is reached.

The result is reported against the decision bands on slide 11, with every figure as k/n and its interval, whatever it turns out to be.

If it passes: fix the real bugs, reword the ambiguities and add the missing tests, each signed off by its owner; then layer 1 in CI, and layer 2 periodically on what changed only: `gate --since REF`.

HONESTLY

Limitations

- **Not a qualified tool.** It is not qualified under ISO 26262-8 (clause 11). Its output is a review aid; every finding is signed off by the requirement owner or the safety reviewer.
- It **does not run tests** and does not check meaning. Bugs come from Copilot's judgement, and a person checks every one.
- Drift is measured from the first `sync` on; earlier drift is layer 2's job.
- `clarify` questions can be guesses: for “clause without case”, 7 of 20 were real (95% interval about 18–57%; 11 of 20 read leniently), measured on the tool's own corpus, not on automotive text (ADR-0075).
- Code signals (`TODO`, empty `catch`, magic numbers) appear only for files no requirement claims.
- Copilot's context is limited: large families are split, and interactions between the parts can be missed.